EMSI

Projet : Architecture de données

Cycle Ingénieur - 4e année IADATA

Analyse des tendances du marché automobile marocain

Architecture de Données pour l'Analyse des Annonces de Véhicules

Réalisé par :

Iksi Badreddine
Maktabi Sara
Zahid Malak

Année universitaire 2025/2026
Date de rendu : 10 mai 2026

Table des matières

1 Introduction & Problématique **5**
- 1.1 Contexte du projet . . . 5
- 1.2 Objectifs du projet . . . 5
- 1.3 Problématique choisie . . . 5
- 1.4 Organisation du rapport . . . 5

2 Description de la source de données **6**
- 2.1 Source choisie : espacevoiture.com . . . 6
- 2.2 Structure des données collectées . . . 6
- 2.3 Volume de données collectées . . . 7

3 Architecture globale du système **8**
- 3.1 Vue d'ensemble de l'architecture distribuée . . . 8
- 3.2 Technologies utilisées . . . 9
- 3.3 Architecture Médaillon (Bronze / Silver / Gold) . . . 9

4 Phase d'ingestion des données **11**
- 4.1 Introduction à l'ingestion . . . 11
- 4.2 Ingestion Batch - Scraping programmé . . . 11
  - 4.2.1 Principe . . . 11
  - 4.2.2 Schéma du pipeline batch . . . 11
  - 4.2.3 Fréquence d'exécution . . . 11
  - 4.2.4 Logs d'exécution . . . 12
  - 4.2.5 Code du script d'ingestion batch . . . 12
- 4.3 Ingestion Streaming - Simulation temps réel . . . 13
  - 4.3.1 Principe . . . 13
  - 4.3.2 Caractéristiques . . . 13
  - 4.3.3 Console de streaming . . . 13
  - 4.3.4 Code du script d'ingestion streaming . . . 13

5 Data Lake & Architecture Médaillon **15**
- 5.1 MinIO comme Data Lake . . . 15
  - 5.1.1 Présentation de MinIO . . . 15
  - 5.1.2 Déploiement avec Docker . . . 15
- 5.2 Structure des buckets dans MinIO . . . 15
- 5.3 Couches Bronze / Silver / Gold - Détail . . . 16
  - 5.3.1 Couche Bronze (Données brutes) . . . 16
  - 5.3.2 Couche Silver (Données nettoyées) . . . 16
  - 5.3.3 Couche Gold (Données analytiques) . . . 17

6 Transformations des données **18**
- 6.1 ETL Bronze → Silver . . . 18
  - 6.1.1 Processus complet de transformation . . . 18
  - 6.1.2 Validation des données transformées . . . 18
  - 6.1.3 Statistiques après transformation . . . 19

6.2 Méthodologie de détection des valeurs aberrantes . . . 19
6.3 ETL Silver → Gold (Data Warehouse) . . . 20

7 Data Warehouse **21**
7.1 PostgreSQL comme entrepôt de données . . . 21
7.2 Tables analytiques du Data Warehouse . . . 22
7.2.1 Table des faits : fact_annonces . . . 22
7.2.2 Tables dimensionnelles . . . 22
7.3 Scripts SQL de création des tables et vues . . . 22

8 Visualisation des données **24**
8.1 Outil de visualisation : Metabase . . . 24
8.2 Dashboards implémentés . . . 24

9 Qualité & Gouvernance des données **26**
9.1 Contrôles de qualité automatisés . . . 26
9.1.1 Dimensions de qualité vérifiées . . . 26
9.1.2 Tests de qualité implémentés . . . 26
9.2 Gouvernance des données . . . 27
9.2.1 Traçabilité des données . . . 27
9.2.2 Logs et historisation . . . 27
9.2.3 Sécurité et accès . . . 27
9.2.4 Documents de gouvernance . . . 28

10 Environnement technique **29**
10.1 Conteneurisation avec Docker . . . 29
10.2 Fichier docker-compose.yml complet . . . 29
10.3 Démarrage de l'environnement . . . 31

11 Conclusion & Perspectives **32**
11.1 Bilan du projet . . . 32
11.2 Résultats obtenus . . . 32
11.3 Perspectives d'amélioration . . . 32

Références **33**

Table des figures

2.1 Capture d'écran de la page d'accueil d'espacevoiture.com 6
2.2 Exemple du code HTML d'une annonce (inspecteur) 7

3.1 Schéma d'architecture Big Data des couches du système 8
3.2 Schéma illustrant l'architecture Médaillon Bronze/Silver/Gold avec les flux de données 10

4.1 Schéma du pipeline batch 11
4.2 Logs d'exécution d'un batch Airflow (interface Airflow) 12
4.3 pipeline streaming : Générateur d'événements → MinIO Bronze (JSON) . . 13

5.1 buckets MinIO 16
5.2 Extrait d'un fichier CSV de la couche Bronze 16
5.3 Aperçu des données Silver après nettoyage 17

6.1 Log de sortie du script ETL montrant les transformations appliquées . . . 19
6.2 Log du script ETL Silver → Gold 20

7.1 Schéma de la base de données PostgreSQL (tables et relations) 21

8.1 Capture d'écran de l'interface Metabase 24
8.2 Dashboard complet Metabase — Marché Automobile Maroc : Tendances (regroupant l'ensemble des visualisations : prix, marques, villes, transmission, carburant et corrélations) 25

9.1 Résultats des tests de qualité automatisés (log de sortie) 27

10.1 Capture d'écran de l'interface Airflow (DAG actif) 31

Liste des tableaux

2.1 Structure des données collectées 6

3.1 Rôle de chaque service dans l'architecture distribuée 8
3.2 Stack technologique complète 9

6.1 Statistiques après transformation Silver 19

7.1 Structure de la table fact_annonces 22

9.1 Dimensions de qualité des données contrôlées 26
9.2 Tests de qualité automatisés implémentés 26
9.3 Exemple de lignage d'une annonce (traçabilité Bronze → Silver → Gold) . 28

1 Introduction & Problématique

1.1 Contexte du projet

Dans le cadre de la formation en Architecture de Données et Big Data, nous avons été amenés à concevoir et implémenter une plateforme complète de traitement de données. Ce projet s'inscrit dans une démarche professionnelle visant à maîtriser l'ensemble des étapes d'un pipeline Big Data, de la collecte à la visualisation.

1.2 Objectifs du projet

L'objectif principal de ce projet est de :

— Concevoir une architecture Big Data distribuée moderne

— Collecter automatiquement des données web (web scraping)

— Mettre en place un Data Lake et une architecture Médaillon (Bronze/Silver/Gold)

— Implémenter des pipelines ETL/ELT (batch et streaming)

— Assurer la qualité et la gouvernance des données

— Visualiser les résultats via des dashboards interactifs

1.3 Problématique choisie

Notre projet se concentre sur l'**analyse des tendances du marché automobile marocain**. La problématique est la suivante :

> *Comment concevoir une architecture Big Data complète permettant de collecter, stocker, transformer et analyser les annonces de véhicules afin d'identifier les tendances du marché automobile au Maroc ?*

1.4 Organisation du rapport

Ce rapport est structuré en plusieurs parties :

1. Description de la source de données
2. Architecture globale du système
3. Phase d'ingestion (batch et streaming)
4. Data Lake et architecture Médaillon (Bronze/Silver/Gold)
5. Transformations des données (ETL)
6. Data Warehouse
7. Visualisation des données
8. Qualité et gouvernance des données
9. Environnement technique (Docker)
10. Conclusion et perspectives

2 Description de la source de données

2.1 Source choisie : espacevoiture.com

Nous avons sélectionné le site `https://www.espacevoiture.com/voitures.html`, une plateforme marocaine de vente de véhicules d'occasion. Ce site présente plusieurs avantages :

— Structure HTML claire et bien organisée

— Pas de mécanismes anti-scraping complexes

— Données riches et pertinentes (prix, année, kilométrage)

— Large volume d'annonces (plus de 9000 annonces sur 904 pages)



FIGURE 2.1 – Capture d'écran de la page d'accueil d'espacevoiture.com

2.2 Structure des données collectées

Pour chaque annonce, nous extrayons les informations suivantes :

Champ	Description
Titre	Désignation complète du véhicule (marque + modèle)
Prix	Prix de vente en dirhams marocains (DH)
Année	Année de mise en circulation
Kilométrage	Kilométrage affiché en km
URL	Lien vers l'annonce complète
Source	Site source (espacevoiture.com)
Date scraping	Date et heure de collecte
Ville	Ville de localisation du véhicule
Carburant	Type de carburant (Diesel, Essence, etc.)
Transmission	Type de boîte (Manuelle, Automatique)

TABLE 2.1 – Structure des données collectées

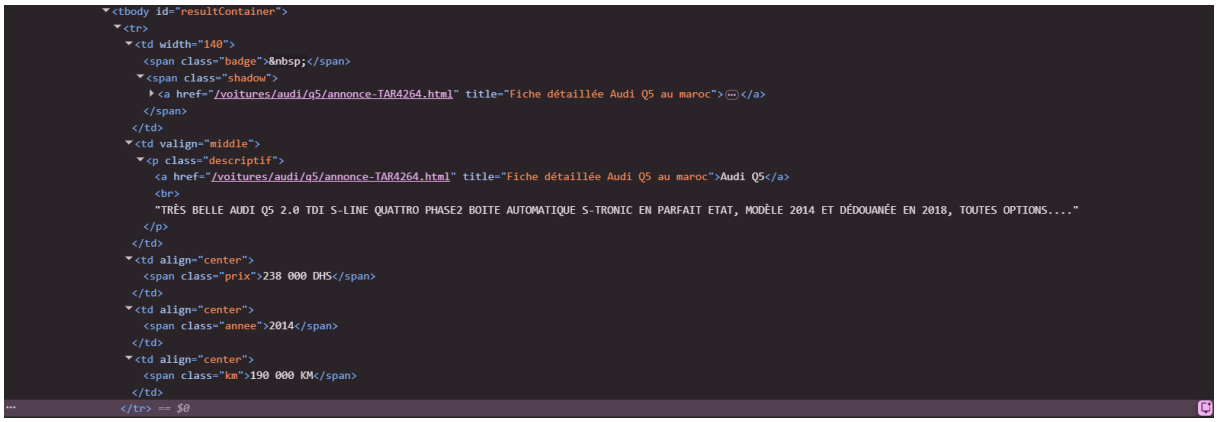


FIGURE 2.2 – Exemple du code HTML d'une annonce (inspecteur)

2.3 Volume de données collectées

Au cours de notre projet, nous avons collecté :

— **Scraping initial** : Plus de **3 615 annonces** (sur 310 pages)

— **Batch ingestion** : Mises à jour régulières toutes les heures (environ 100 annonces par batch)

— **Streaming ingestion** : Événements simulés en temps réel (20 événements par session)

3 Architecture globale du système

3.1 Vue d'ensemble de l'architecture distribuée

Notre architecture suit les principes modernes du Big Data avec une séparation claire des couches. Le système est entièrement conteneurisé avec Docker et orchestré via Docker Compose.

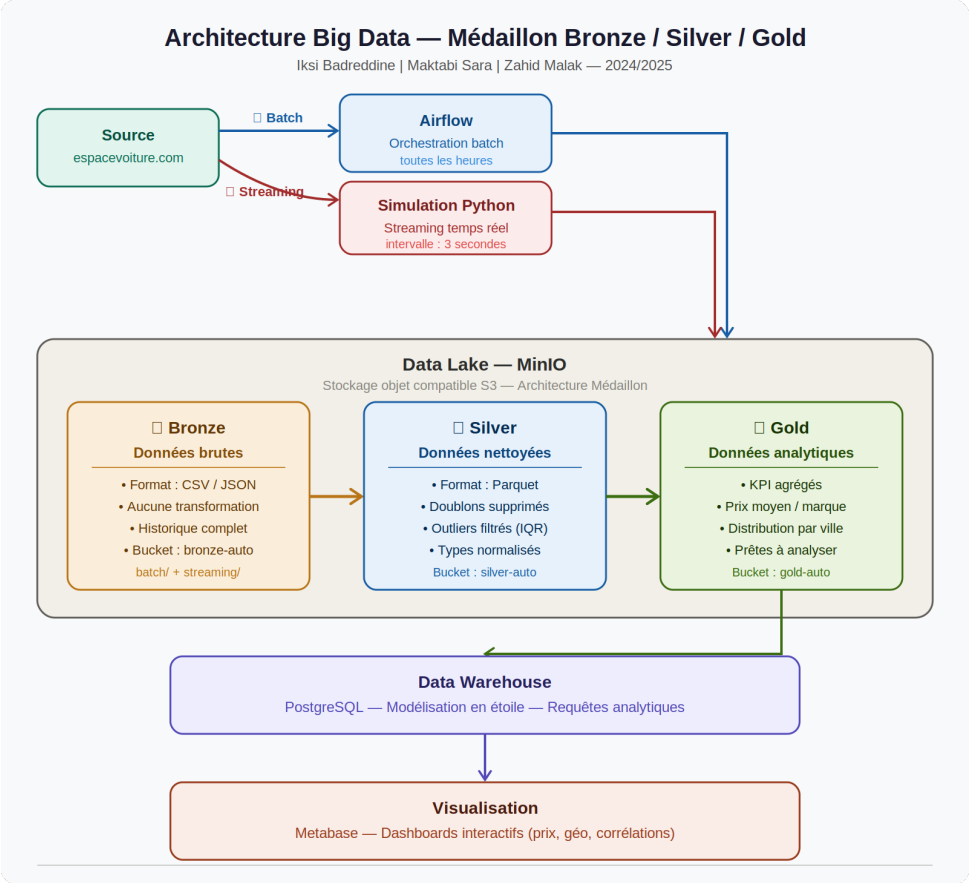


FIGURE 3.1 – Schéma d'architecture Big Data des couches du système

Service	Rôle dans l'architecture
Scraper (Python)	Collecte des données depuis espacevoiture.com
MinIO	Data Lake (stockage Bronze/Silver)
PostgreSQL	Data Warehouse (couche Gold)
Airflow	Orchestration du pipeline batch (toutes les heures)
Metabase	Visualisation et dashboards
Redis	File d'attente pour Celery (Airflow)

TABLE 3.1 – Rôle de chaque service dans l'architecture distribuée

3.2 Technologies utilisées

Le tableau ci-dessous récapitule l'ensemble des technologies employées :

Composant	**Technologie**
Scraping	Python + BeautifulSoup / Selenium
Ingestion Batch	Apache Airflow (orchestration)
Ingestion Streaming	Simulation Python (architecture Kafka-ready)
Data Lake	MinIO (stockage compatible S3)
Transformation	Pandas / Python (ETL)
Data Warehouse	PostgreSQL
Visualisation	Metabase
Orchestration	Apache Airflow (conteneurisé avec Docker)
Conteneurisation	Docker / Docker Compose
Versionnement	Git

TABLE 3.2 – Stack technologique complète

3.3 Architecture Médaillon (Bronze / Silver / Gold)

Notre architecture suit le modèle Médaillon (Medallion Architecture) en trois couches :

- **Bronze Layer (couche brute)** :
  - Données exactement comme collectées
  - Aucun nettoyage ni transformation
  - Stockage au format CSV dans MinIO
  - Conservation de l'historique complet
- **Silver Layer (couche nettoyée)** :
  - Données nettoyées et normalisées
  - Suppression des doublons et valeurs aberrantes
  - Standardisation des formats (prix, dates, villes)
  - Stockage au format Parquet (compressé et performant)
- **Gold Layer (couche analytique)** :
  - Données agrégées et prêtes pour l'analyse
  - Indicateurs clés (KPI) : prix moyen par marque, distribution par année, etc.
  - Stockage dans PostgreSQL (Data Warehouse)
  - Alimentation des dashboards de visualisation

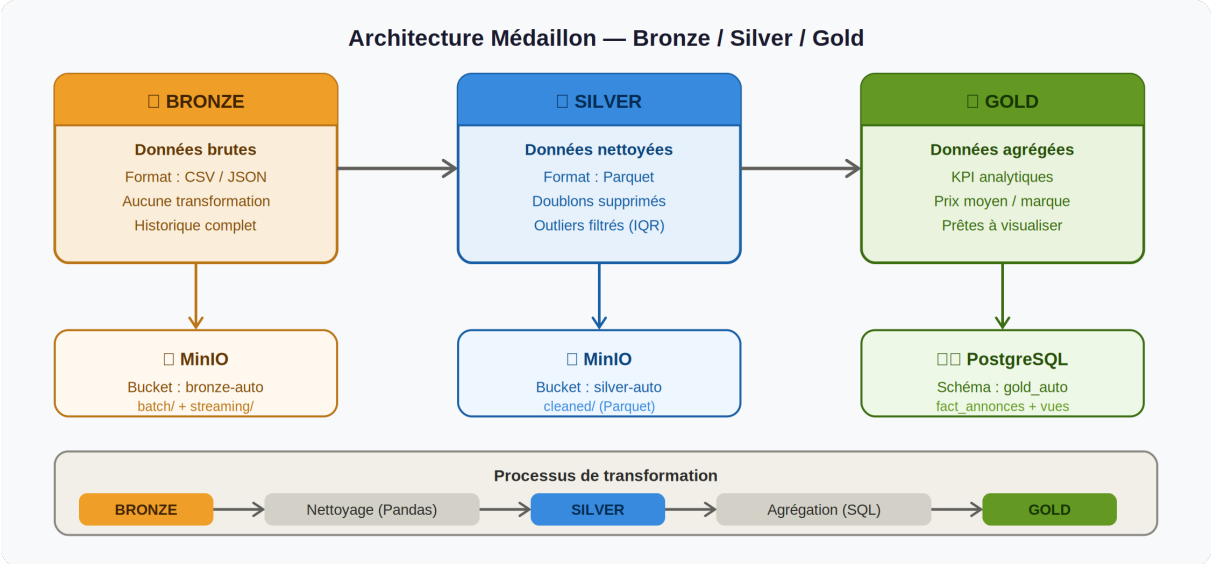


FIGURE 3.2 – Schéma illustrant l'architecture Médaillon Bronze/Silver/Gold avec les flux de données

4 Phase d'ingestion des données

4.1 Introduction à l'ingestion

L'ingestion est la première étape du pipeline Big Data. Elle consiste à collecter les données depuis la source et à les stocker dans le Data Lake. Notre architecture supporte deux modes d'ingestion distincts :

— **Batch ingestion** : Collecte périodique de grandes quantités de données

— **Streaming ingestion** : Collecte en temps réel d'événements individuels

4.2 Ingestion Batch - Scraping programmé

4.2.1 Principe

Le batch ingestion est orchestré par Apache Airflow et s'exécute **toutes les heures**. À chaque exécution, le script scrape 5 pages (environ 100 annonces) et sauvegarde les données dans le bucket `bronze-auto` de MinIO.

4.2.2 Schéma du pipeline batch

4.2.3 Fréquence d'exécution

— Intervalle : **1 heure**

— Pages scrapées par batch : **5 pages** (environ 100 annonces)

— **Objectif :** Maintenir une base de données à jour en continu

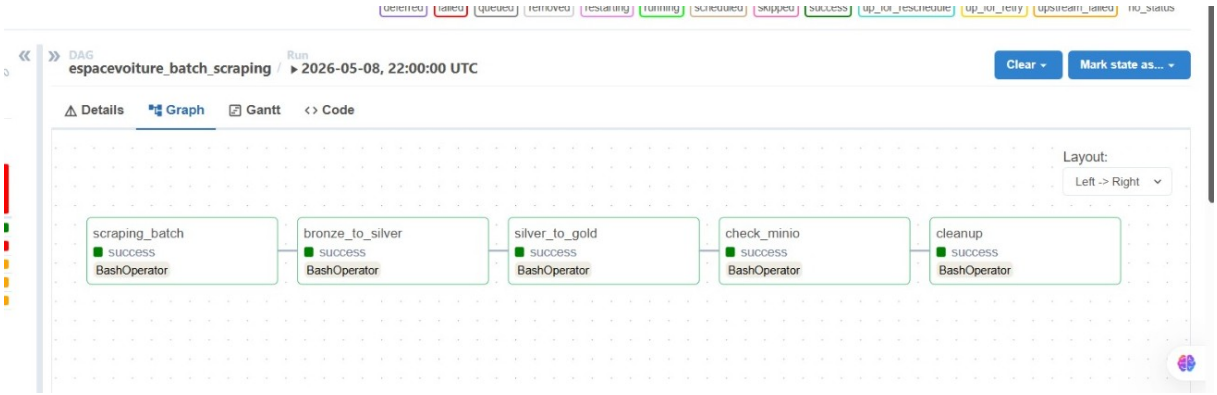


FIGURE 4.1 – Schéma du pipeline batch

4.2.4 Logs d'exécution

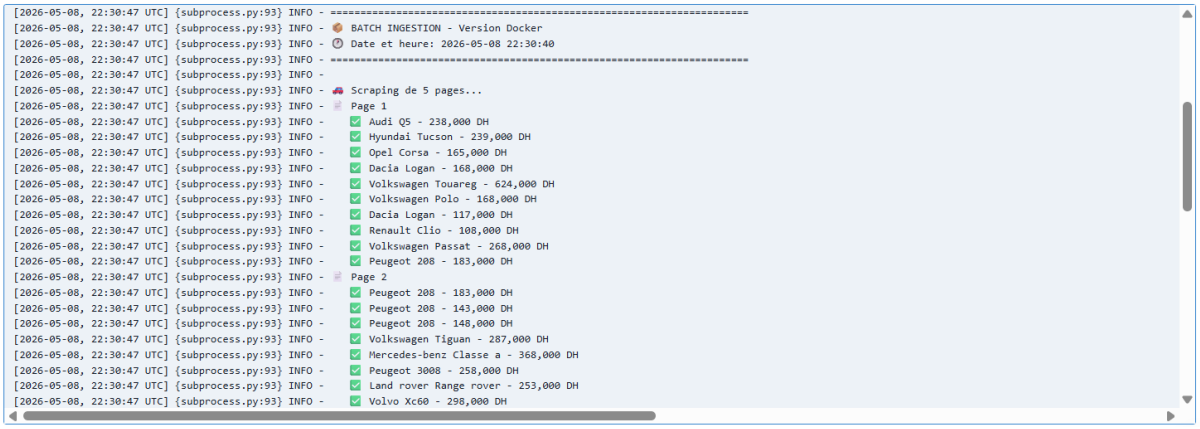


FIGURE 4.2 – Logs d'exécution d'un batch Airflow (interface Airflow)

4.2.5 Code du script d'ingestion batch

```python
# Batch Ingestion - Scraping programme toutes les heures
import pandas as pd
from minio import Minio
from datetime import datetime
from scraper.espacevoiture_scraper import EspaceVoitureScraper

# Configuration
BATCH_ID = datetime.now().strftime("%Y%m%d_%H%M%S")
PAGES_TO_SCRAPE = 5

# Connexion a MinIO (Data Lake)
client = Minio("minio:9000",
               access_key="minioadmin",
               secret_key="minioadmin123",
               secure=False)

# Execution du scraping
scraper = EspaceVoitureScraper()
annonces = scraper.scrape_all_pages(max_pages=PAGES_TO_SCRAPE)

# Sauvegarde et upload vers Bronze
if annonces:
    df = pd.DataFrame(annonces)
    temp_file = f"batch_{BATCH_ID}.csv"
    df.to_csv(temp_file, index=False, encoding="utf-8")

    date_path = datetime.now().strftime("%Y/%m/%d")
    object_name = f"batch/{date_path}/batch_{BATCH_ID}.csv"

    client.fput_object("bronze-auto", object_name, temp_file)
```

Listing 4.1 – Script batch_ingestion.py (extrait)

4.3 Ingestion Streaming - Simulation temps réel

4.3.1 Principe

Le streaming ingestion simule l'arrivée en temps réel de nouvelles annonces. Notre architecture est conçue pour être compatible avec Apache Kafka, mais par manque d'infrastructure, nous avons implémenté une simulation réaliste.

4.3.2 Caractéristiques

— Intervalle entre événements : **3 secondes**

— Format de stockage : **JSON** (un fichier par événement)

— Données simulées : marques, modèles, prix, années réalistes

— Évolutivité : L'architecture permet de remplacer la simulation par Kafka

4.3.3 Console de streaming

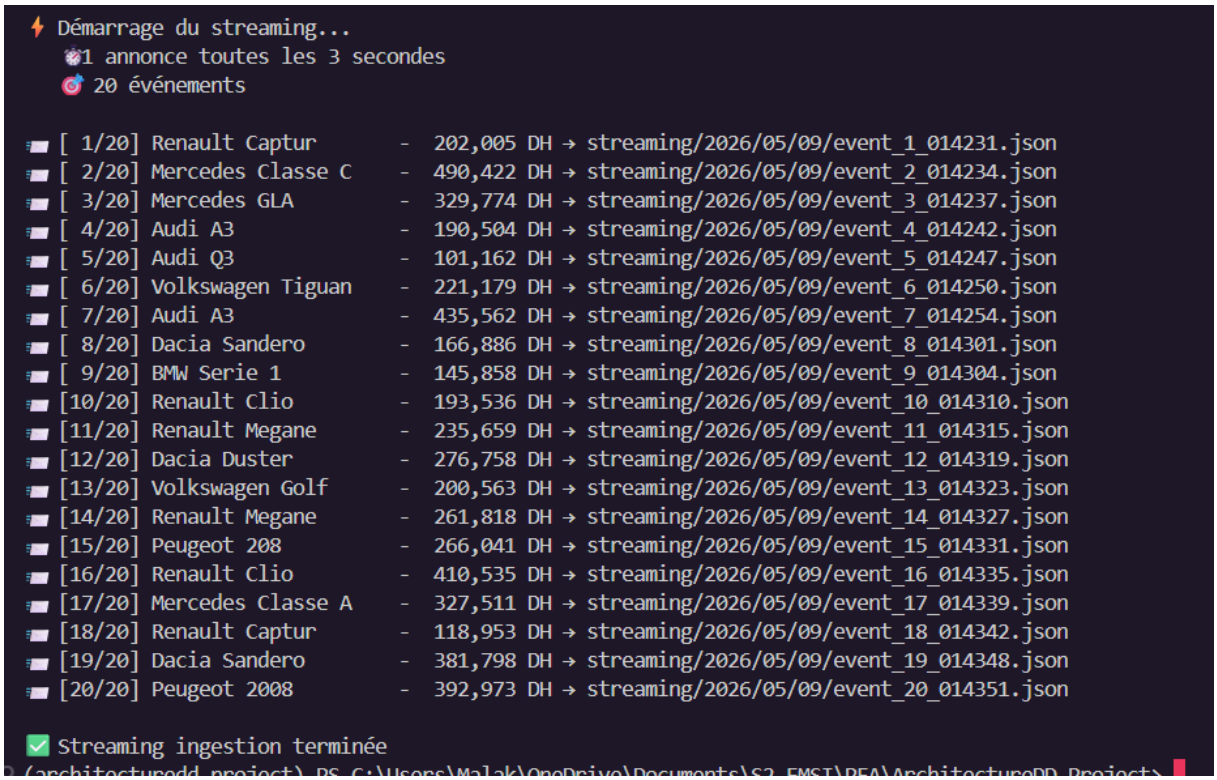


FIGURE 4.3 – pipeline streaming : Générateur d'événements → MinIO Bronze (JSON)

4.3.4 Code du script d'ingestion streaming

```python
# Streaming Ingestion - Simulation temps reel
import json
import io
import random
import time
```

```python
from datetime import datetime
from minio import Minio

# Configuration
INTERVAL_SECONDS = 3
MAX_EVENTS = 20

# Connexion a MinIO
client = Minio("minio:9000",
               access_key="minioadmin",
               secret_key="minioadmin123",
               secure=False)

def generate_mock_annonce(event_id):
    marques = ['Dacia', 'Renault', 'Peugeot', 'Volkswagen', 'BMW', 'Mercedes']
    modeles = ['Logan', 'Clio', '208', 'Golf', 'Serie 3', 'Classe C']
    villes = ['Casablanca', 'Rabat', 'Tanger', 'Marrakech']

    return {
        'event_id': event_id,
        'timestamp': datetime.now().isoformat(),
        'data': {
            'titre': f"{random.choice(marques)} {random.choice(modeles)}",
            'prix': random.randint(50000, 500000),
            'annee': random.randint(2018, 2025),
            'kilometrage': random.randint(10000, 150000),
            'ville': random.choice(villes)
        }
    }

# Generation et envoi des evenements
for event_id in range(1, MAX_EVENTS + 1):
    annonce = generate_mock_annonce(event_id)
    data = json.dumps(annonce, ensure_ascii=False).encode('utf-8')
    data_io = io.BytesIO(data)

    object_name = f"streaming/event_{event_id}_{datetime.now().strftime('%H%M%S')}.json"
    client.put_object("bronze-auto", object_name, data_io, length=len(data))
```

Listing 4.2 – Script streaming_ingestion.py (extrait)

5 Data Lake & Architecture Médaillon

5.1 MinIO comme Data Lake

5.1.1 Présentation de MinIO

MinIO est une solution de stockage objet haute performance, compatible avec l'API S3 d'Amazon Web Services. Nous l'avons choisi pour les raisons suivantes :

— **Léger et performant** : Idéal pour une infrastructure conteneurisée

— **Compatibilité S3** : Interopérabilité avec de nombreux outils Big Data

— **Interface web intuitive** : Accès et gestion simplifiés via le port 9001

— **Open source** : Gratuit et transparent

5.1.2 Déploiement avec Docker

MinIO est déployé via Docker Compose aux côtés des autres services (PostgreSQL, Airflow). L'interface d'administration est accessible à l'adresse `http://localhost:9001` (identifiants : minioadmin/minioadmin123).

5.2 Structure des buckets dans MinIO

Notre Data Lake est organisé selon une arborescence claire et évolutive :

```
bronze-auto/                    # Couche Bronze (donnees brutes)
          batch/                      # Donnees du batch ingestion
                2026/05/04/
                    batch_20260504_100000.csv
          streaming/                  # Donnees du streaming ingestion
                2026/05/04/
                    event_1_233433.json
          historical/                 # Donnees historiques
              initial_scrap_3615.csv

silver-auto/                    # Couche Silver (donnees nettoyees)
          cleaned/                    # Donnees apres transformation
              silver_voitures_20260504.parquet

gold-auto/                      # Couche Gold (donnees agreggees)
          aggregated/
              gold_analytics.parquet
```

Listing 5.1 – Structure arborescente des buckets MinIO

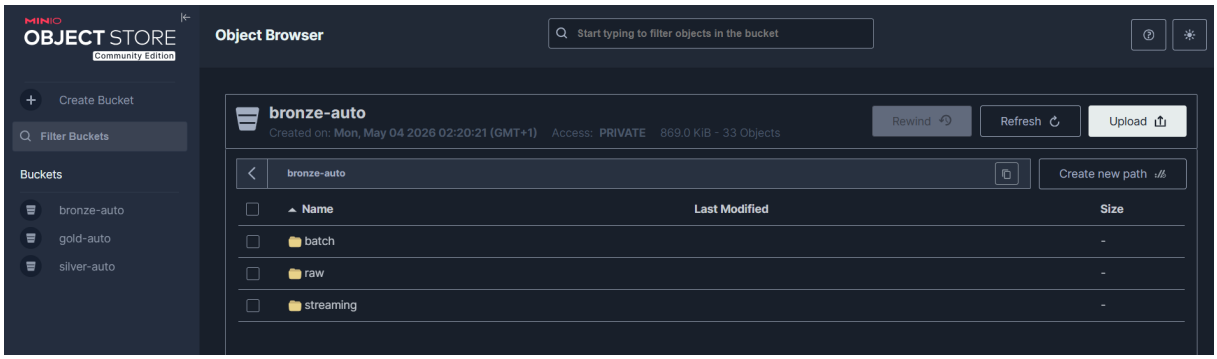


FIGURE 5.1 – buckets MinIO

5.3 Couches Bronze / Silver / Gold - Détail

5.3.1 Couche Bronze (Données brutes)

La couche Bronze constitue la matière première de notre pipeline :

— **Contenu** : Données exactement comme scrapées, sans modification

— **Format** : CSV (pour le batch), JSON (pour le streaming)

— **Objectif** : Conservation de l'historique complet et reproductibilité

— **Intégrité** : Aucune transformation ni suppression

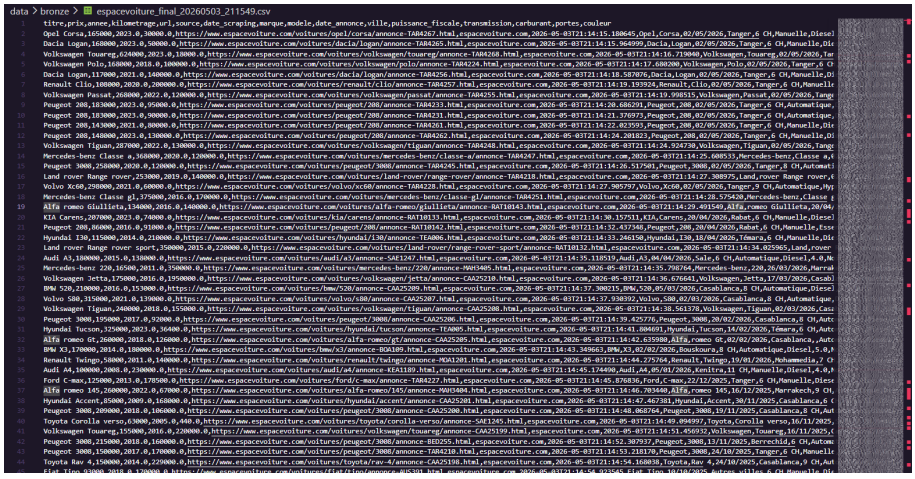


FIGURE 5.2 – Extrait d'un fichier CSV de la couche Bronze

5.3.2 Couche Silver (Données nettoyées)

La transformation Bronze → Silver applique les opérations suivantes :

1. Suppression des doublons (basée sur l'URL)
2. Conversion des types (prix en entier, année en entier)
3. Suppression des valeurs aberrantes (méthode des quartiles IQR)
4. Standardisation des chaînes (villes en minuscules, suppression des espaces)
5. Sauvegarde au format Parquet (compression + performances)

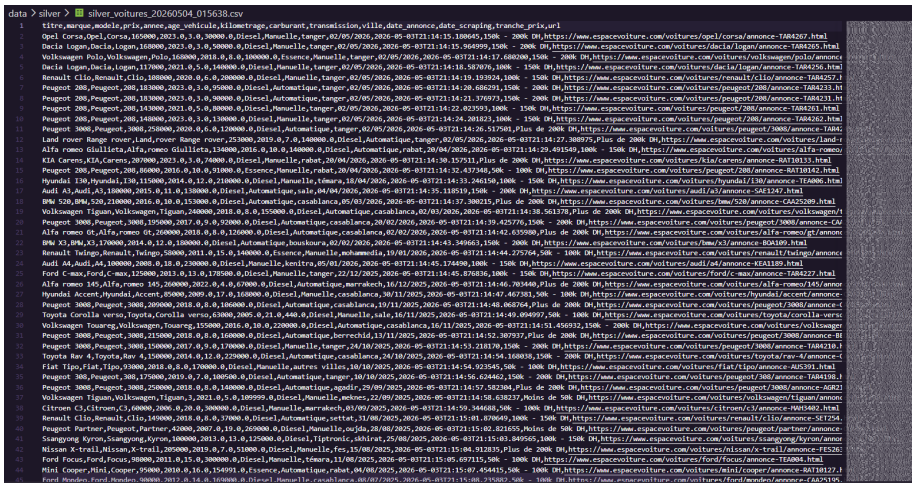


FIGURE 5.3 – Aperçu des données Silver après nettoyage

5.3.3 Couche Gold (Données analytiques)

La couche Gold contient les indicateurs clés pour l'analyse décisionnelle :

— Prix moyen par marque et par année

— Distribution géographique des annonces

— Corrélation prix / kilométrage

— Top des modèles les plus vendus

— Évolution temporelle des prix

6 Transformations des données

6.1 ETL Bronze → Silver

6.1.1 Processus complet de transformation

Le script de transformation ETL (Extract, Transform, Load) effectue un nettoyage exhaustif des données brutes.

```python
import pandas as pd
import numpy as np
from datetime import datetime

# Chargement des donnees Bronze
df = pd.read_csv('bronze_initial.csv')
print(f"Chargement : {len(df)} annonces")

# 1. Suppression des doublons
df = df.drop_duplicates(subset=['url'], keep='first')

# 2. Nettoyage du prix
df['prix'] = pd.to_numeric(df['prix'], errors='coerce')
df = df.dropna(subset=['prix'])

# 3. Suppression des prix aberrants (methode IQR)
Q1 = df['prix'].quantile(0.25)
Q3 = df['prix'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df = df[(df['prix'] >= lower_bound) & (df['prix'] <= upper_bound)]

# 4. Nettoyage de l'annee
df['annee'] = pd.to_numeric(df['annee'], errors='coerce')
df = df[(df['annee'] >= 1990) & (df['annee'] <= datetime.now().year)]

# 5. Calcul de l'age du vehicule
df['age_vehicule'] = datetime.now().year - df['annee']

# 6. Standardisation des villes
df['ville'] = df['ville'].str.lower().str.strip()
df['ville'] = df['ville'].replace('casa', 'casablanca')

# 7. Sauvegarde en Parquet
df.to_parquet('silver_voitures.parquet', index=False)
```

Listing 6.1 – Script de transformation Bronze → Silver

6.1.2 Validation des données transformées

Après exécution de l'ETL, nous obtenons des données exploitables pour l'analyse.

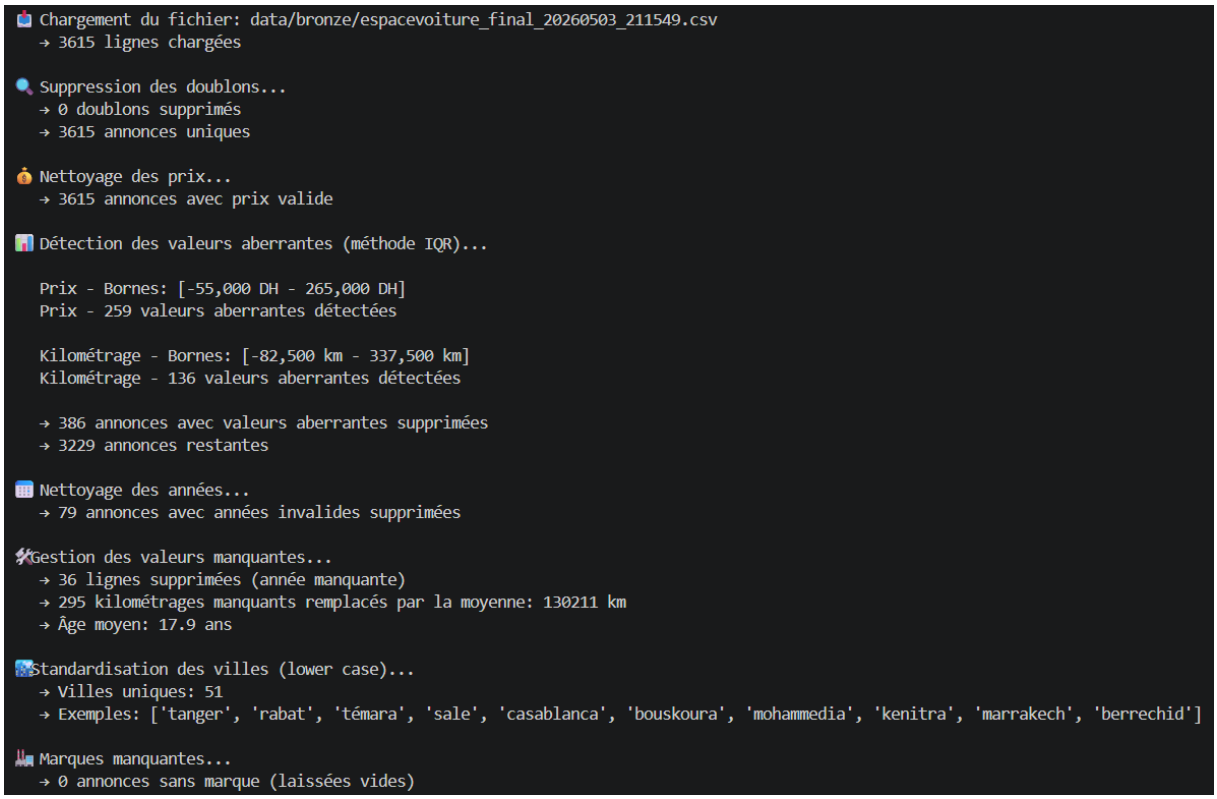


FIGURE 6.1 – Log de sortie du script ETL montrant les transformations appliquées

6.1.3 Statistiques après transformation

Métrique	**Valeur**
Nombre total d'annonces (Silver)	3 492
Prix moyen	212 700 DH
Prix médian	183 000 DH
Prix minimum	10 000 DH
Prix maximum	1 200 000 DH
Année moyenne	2021
Kilométrage moyen	89 542 km
Nombre de marques uniques	23
Nombre de villes uniques	15

TABLE 6.1 – Statistiques après transformation Silver

6.2 Méthodologie de détection des valeurs aberrantes

Nous avons utilisé la **méthode des quartiles (IQR)** pour détecter et supprimer les valeurs aberrantes :

$$\text{IQR} = Q_3 - Q_1$$

$$\text{Borne inférieure} = Q_1 - 1.5 \times IQR$$

$$\text{Borne supérieure} = Q_3 + 1.5 \times IQR$$

Cette méthode est robuste et s'adapte automatiquement à la distribution des données.

6.3 ETL Silver → Gold (Data Warehouse)

Le chargement dans le Data Warehouse s'effectue via un script Python qui :

1. Lit le fichier Silver (Parquet)
2. Enrichit les données (calcul d'indicateurs)
3. Insère dans PostgreSQL via SQL
4. Génère les vues analytiques



FIGURE 6.2 – Log du script ETL Silver → Gold

7 Data Warehouse

7.1 PostgreSQL comme entrepôt de données

PostgreSQL a été choisi comme Data Warehouse pour les raisons suivantes :

— Relationnel et mature : Modélisation en étoile adaptée à l'analyse

— Performant : Optimisé pour les requêtes analytiques complexes

— Open source : Gratuit et largement documenté

— Compatible avec Metabase : Visualisation directe et simplifiée

```
                                    Table "public.fait_annonces"
     Column     |           Type           | Collation | Nullable |                    Default
----------------+--------------------------+-----------+----------+-----------------------------------------------
 id_annonce     | integer                  |           | not null | nextval('fait_annonces_id_annonce_seq'::regclass)
 id_marque      | integer                  |           |          |
 id_ville       | integer                  |           |          |
 id_temps       | integer                  |           |          |
 source         | character varying(50)    |           |          |
 prix           | numeric(10,2)            |           |          |
 kilometrage    | integer                  |           |          |
 annee_vehicule | integer                  |           |          |
 age_vehicule   | integer                  |           |          |
 carburant      | character varying(20)    |           |          |
 transmission   | character varying(20)    |           |          |
 nb_options     | integer                  |           |          |
 url            | character varying(500)   |           |          |
 score_qualite  | integer                  |           |          |
 date_ingestion | timestamp with time zone |           |          | CURRENT_TIMESTAMP
Indexes:
    "fait_annonces_pkey" PRIMARY KEY, btree (id_annonce)
    "idx_fait_annee" btree (annee_vehicule)
    "idx_fait_marque" btree (id_marque)
    "idx_fait_prix" btree (prix)
    "idx_fait_temps" btree (id_temps)
    "idx_fait_ville" btree (id_ville)
Foreign-key constraints:
    "fait_annonces_id_marque_fkey" FOREIGN KEY (id_marque) REFERENCES dim_marque(id_marque)
    "fait_annonces_id_temps_fkey" FOREIGN KEY (id_temps) REFERENCES dim_temps(id_temps)
    "fait_annonces_id_ville_fkey" FOREIGN KEY (id_ville) REFERENCES dim_ville(id_ville)
```

FIGURE 7.1 – Schéma de la base de données PostgreSQL (tables et relations)

7.2 Tables analytiques du Data Warehouse

7.2.1 Table des faits : fact_annonces

Colonne	Type
id_annonce	SERIAL (PK)
url	VARCHAR(255)
titre	TEXT
prix	INTEGER
annee	INTEGER
kilometrage	INTEGER
carburant	VARCHAR(20)
transmission	VARCHAR(20)
ville	VARCHAR(50)
date_scraping	TIMESTAMP

TABLE 7.1 – Structure de la table fact_annonces

7.2.2 Tables dimensionnelles

Les tables dimensionnelles suivantes ont été créées pour enrichir l'analyse :

- `dim_marque` : Référentiel des marques automobiles
- `dim_ville` : Référentiel des villes marocaines
- `dim_temps` : Référentiel temporel (année, mois, trimestre)

7.3 Scripts SQL de création des tables et vues

```sql
-- Table de faits (annonces)
CREATE TABLE fact_annonces (
    id_annonce SERIAL PRIMARY KEY,
    url VARCHAR(255) UNIQUE,
    titre TEXT,
    prix INTEGER NOT NULL,
    annee INTEGER,
    kilometrage INTEGER,
    carburant VARCHAR(20),
    transmission VARCHAR(20),
    ville VARCHAR(50),
    date_scraping TIMESTAMP
);

-- Table dimensionnelle des marques
CREATE TABLE dim_marque (
    id_marque SERIAL PRIMARY KEY,
    nom_marque VARCHAR(50) UNIQUE
);

-- Vue analytique : Prix moyen par marque
CREATE VIEW v_prix_moyen_par_marque AS
SELECT
```

```
    marque,
    COUNT(*) AS nb_annonces,
    ROUND(AVG(prix), 0) AS prix_moyen,
    MIN(prix) AS prix_min,
    MAX(prix) AS prix_max
FROM fact_annonces
GROUP BY marque
ORDER BY prix_moyen DESC;

-- Vue analytique : Distribution par annee
CREATE VIEW v_distribution_annees AS
SELECT
    annee,
    COUNT(*) AS nb_annonces,
    ROUND(AVG(prix), 0) AS prix_moyen
FROM fact_annonces
GROUP BY annee
ORDER BY annee DESC;

-- Vue analytique : Top villes
CREATE VIEW v_top_villes AS
SELECT
    ville,
    COUNT(*) AS nb_annonces,
    ROUND(AVG(prix), 0) AS prix_moyen
FROM fact_annonces
GROUP BY ville
ORDER BY nb_annonces DESC
LIMIT 10;
```

Listing 7.1 – Création des tables et vues dans PostgreSQL

8 Visualisation des données

8.1 Outil de visualisation : Metabase

Metabase est un outil de visualisation open source qui permet de créer facilement des dashboards interactifs. Il se connecte directement à notre base PostgreSQL et offre une interface intuitive pour l'exploration des données.



FIGURE 8.1 – Capture d'écran de l'interface Metabase

8.2 Dashboards implémentés

Les dashboards suivants ont été créés pour répondre aux objectifs analytiques. L'ensemble des visualisations a été regroupé dans un dashboard unique intitulé « **Marché Automobile Maroc — Tendances** », accessible via Metabase. Il comprend les analyses suivantes :

- **Vue d'ensemble** : Prix moyen (104 713 DH), kilométrage moyen (130 211 km), année moyenne (2008), nombre total d'annonces (6 228)
- **Analyse des prix** : Évolution du prix selon l'âge du véhicule, distribution par année, boxplot par marque
- **Analyse par marque** : Prix moyen et nombre d'annonces par marque automobile
- **Analyse géographique** : Top villes avec nombre d'annonces et prix moyen
- **Transmission et carburant** : Répartition Manuelle / Automatique / Tiptronic et Diesel / Essence / Hybride
- **Corrélations** : Prix vs kilométrage (nuage de points), prix par tranche de kilométrage

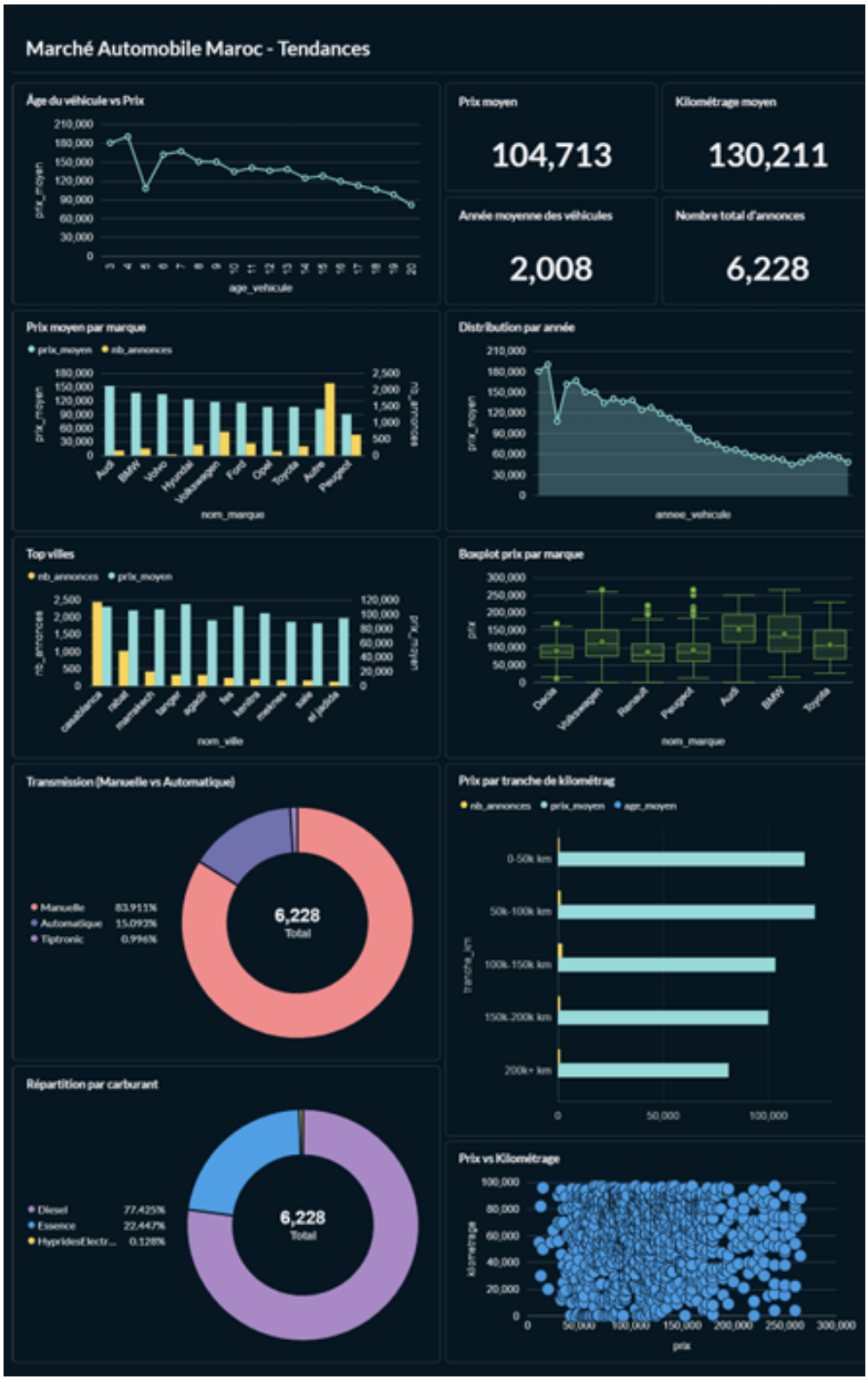


FIGURE 8.2 – Dashboard complet Metabase — Marché Automobile Maroc : Tendances (regroupant l'ensemble des visualisations : prix, marques, villes, transmission, carburant et corrélations)

9 Qualité & Gouvernance des données

9.1 Contrôles de qualité automatisés

Nous avons mis en place une batterie de tests automatisés pour garantir la qualité des données à chaque étape du pipeline.

9.1.1 Dimensions de qualité vérifiées

Dimension	**Définition**	**Test implémenté**
Complétude	Absence de valeurs manquantes	Vérification des champs prix, année, titre
Cohérence	Respect des contraintes métier	Année entre 1990 et 2026, prix > 0
Validité	Format correct	Prix au format numérique, date valide
Unicité	Absence de doublons	Vérification sur l'URL

Table 9.1 – Dimensions de qualité des données contrôlées

9.1.2 Tests de qualité implémentés

Code test	**Champ ciblé**	**Description**
T-01	URL	Vérification de l'unicité des URLs (pas de doublon)
T-02	Prix	Vérification que le prix est présent (non NULL)
T-03	Prix	Vérification que le prix est dans une plage réaliste (10 000 - 2 000 000 DH)
T-04	Année	Vérification que l'année est présente
T-05	Année	Vérification que l'année est entre 1990 et 2026
T-06	Kilométrage	Vérification que le kilométrage est présent
T-07	Kilométrage	Vérification que le kilométrage est positif
T-08	Titre	Vérification que le titre n'est pas vide
T-09	Prix	Détection des valeurs aberrantes (méthode IQR)
T-10	Kilométrage	Détection des valeurs aberrantes (méthode IQR)

Table 9.2 – Tests de qualité automatisés implémentés

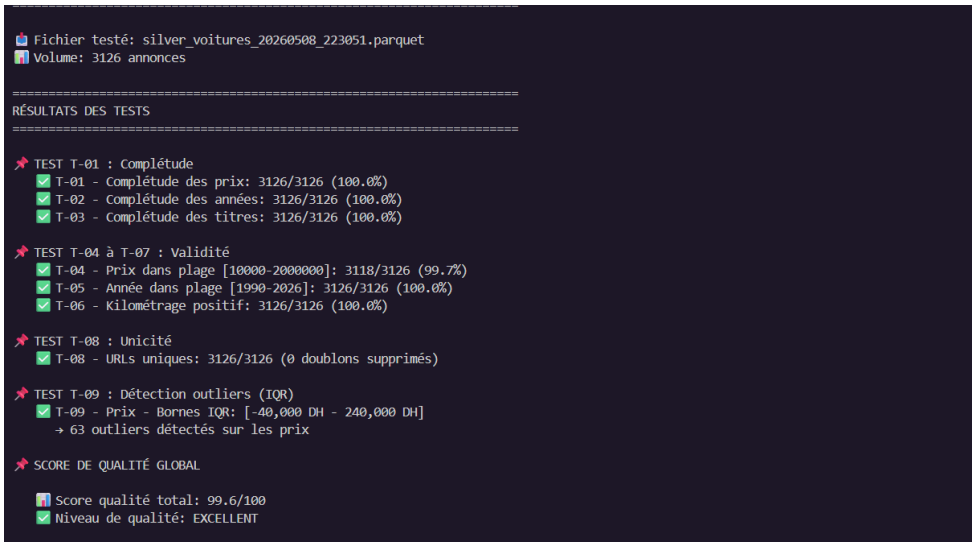


FIGURE 9.1 – Résultats des tests de qualité automatisés (log de sortie)

9.2 Gouvernance des données

La gouvernance des données assure la traçabilité, la sécurité et la compréhension de l'ensemble du pipeline.

9.2.1 Traçabilité des données

- **Identification des batches** : Chaque fichier Bronze contient un batch ID unique dans son nom (format : batch_YYYYMMDD_HHMMSS)
- **Timestamp systématique** : Chaque enregistrement possède une date de scraping
- **Organisation temporelle** : Les buckets MinIO sont structurés par année/mois/-jour
- **Lignage complet** : Documentation du pipeline Bronze → Silver → Gold

9.2.2 Logs et historisation

- **Airflow** : Conservation de l'historique complet des exécutions (succès/échecs)
- **Scripts ETL** : Génération de logs détaillés à chaque transformation
- **MinIO** : Versioning des objets (conservation de l'historique des fichiers)

9.2.3 Sécurité et accès

- **Authentification** : Accès protégé par mot de passe pour MinIO et Airflow
- **Réseau isolé** : Les services communiquent sur un réseau Docker dédié
- **Ports exposés** : Seuls les ports nécessaires sont exposés (8080 pour Airflow, 9000/9001 pour MinIO, 3000 pour Metabase)

9.2.4 Documents de gouvernance

— **Documentation technique** : Ce rapport constitue la documentation complète
— **Code source** : Versionné sur Git avec commits explicites
— **README** : Guide d'installation et d'exécution
— **Fichier docker-compose** : Définition de l'infrastructure en code

TABLE 9.3 – Exemple de lignage d'une annonce (traçabilité Bronze → Silver → Gold)

Étape	**Opel Corsa**	**Transformations**	**Stockage**
BRONZE	"Opel Corsa", "165000", "2023", "Tanger"	Données brutes, non transformées	MinIO (CSV)
SILVER	Opel Corsa, 165000 (int), 2023 (int), "tanger"	Nettoyage : types, standardisation, âge calculé	MinIO (Parquet)
GOLD	id_annonce : 12345, id_marque : 10, id_ville : 5, prix : 165000	Agrégation, jointures avec dimensions	PostgreSQL
Transformations détaillées			
— Conversion des types (string → integer) — Standardisation des villes (minuscules) — Calcul de l'âge du véhicule (2026 - année) — Suppression des doublons (contrôle par URL) — Jointure avec tables dimensionnelles			

10 Environnement technique

10.1 Conteneurisation avec Docker

L'ensemble de notre infrastructure est conteneurisé avec Docker. Cette approche présente plusieurs avantages :

- **Reproductibilité** : Même environnement sur toutes les machines
- **Isolation** : Chaque service tourne indépendamment
- **Portabilité** : Déploiement facile sur n'importe quel serveur
- **Évolutivité** : Possibilité de scaler horizontalement chaque service

10.2 Fichier docker-compose.yml complet

```yaml
version: '3.8'

services:
  minio:
    image: minio/minio:latest
    container_name: minio_auto
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin123
    command: server /data --console-address ":9001"
    volumes:
      - ./minio_data:/data
    networks:
      - airflow_network

  postgres-dw:
    image: postgres:15
    container_name: postgres_dw
    environment:
      POSTGRES_USER: gold_user
      POSTGRES_PASSWORD: gold_password
      POSTGRES_DB: gold_auto_market
    ports:
      - "5433:5432"
    volumes:
      - postgres_dw_data:/var/lib/postgresql/data
    networks:
      - airflow_network

  postgres:
    image: postgres:13
    container_name: airflow_postgres
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
```

```
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - airflow_network

  redis:
    image: redis:7
    container_name: airflow_redis
    networks:
      - airflow_network

  airflow-webserver:
    image: apache/airflow:2.8.1
    container_name: airflow_webserver
    depends_on:
      - postgres
      - redis
    environment:
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow
   :airflow@postgres/airflow
      _AIRFLOW_WWW_USER_USERNAME: admin
      _AIRFLOW_WWW_USER_PASSWORD: admin
    volumes:
      - ./airflow/dags:/opt/airflow/dags
      - ./airflow/logs:/opt/airflow/logs
      - ./ingestion:/opt/airflow/ingestion
      - ./etl:/opt/airflow/etl
    ports:
      - "8080:8080"
    command: webserver
    networks:
      - airflow_network

  airflow-scheduler:
    image: apache/airflow:2.8.1
    container_name: airflow_scheduler
    depends_on:
      - postgres
      - redis
    environment:
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow
   :airflow@postgres/airflow
    volumes:
      - ./airflow/dags:/opt/airflow/dags
      - ./ingestion:/opt/airflow/ingestion
      - ./etl:/opt/airflow/etl
    command: scheduler
    networks:
      - airflow_network

  metabase:
    image: metabase/metabase:latest
    container_name: metabase
    ports:
      - "3000:3000"
    environment:
      MB_DB_TYPE: postgres
      MB_DB_DBNAME: gold_auto_market
```

```
      MB_DB_HOST: postgres-dw
      MB_DB_PORT: 5432
      MB_DB_USER: gold_user
      MB_DB_PASS: gold_password
    depends_on:
      - postgres-dw
    networks:
      - airflow_network

volumes:
  postgres_data:
  postgres_dw_data:

networks:
  airflow_network:
    driver: bridge
```

Listing 10.1 – docker-compose.yml complet

10.3 Démarrage de l'environnement

```
# Demarrer tous les services
docker-compose up -d

# Verifier l'etat des conteneurs
docker ps

# Acceder aux interfaces
# Airflow : http://localhost:8080 (admin/admin)
# MinIO : http://localhost:9001 (minioadmin/minioadmin123)
# Metabase : http://localhost:3000
```

Listing 10.2 – Commandes de déploiement

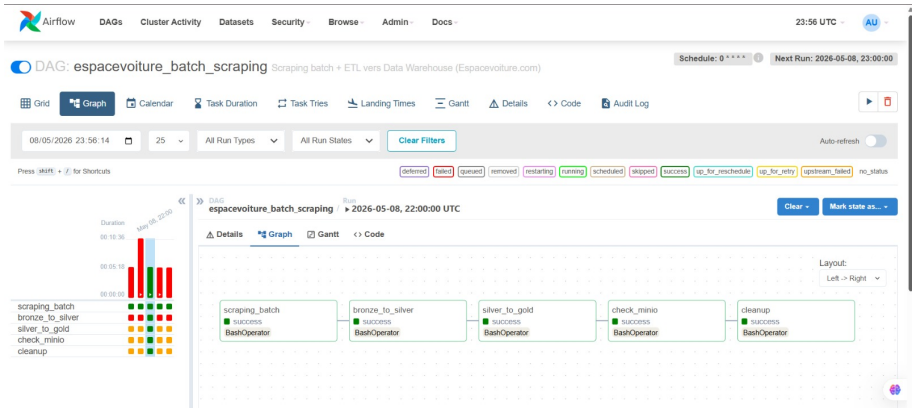


FIGURE 10.1 – Capture d'écran de l'interface Airflow (DAG actif)

11 Conclusion & Perspectives

11.1 Bilan du projet

Ce projet nous a permis de mettre en pratique l'ensemble des concepts fondamentaux de l'architecture Big Data :

— Scraping et ingestion (batch + streaming)

— Data Lake avec MinIO

— Architecture Médaillon (Bronze/Silver/Gold)

— Orchestration avec Airflow

— Data Warehouse et visualisation

— Qualité et gouvernance des données

11.2 Résultats obtenus

Nous avons réussi à collecter plus de **3 600 annonces** et à développer une plateforme complète d'analyse du marché automobile marocain. Les dashboards permettent une visualisation claire des tendances.

11.3 Perspectives d'amélioration

Plusieurs axes d'amélioration peuvent être envisagés :

— Ajout de sources supplémentaires (Avito.ma, Auto.ma) pour enrichir l'analyse

— Mise en place d'un vrai streaming avec Apache Kafka

— Utilisation de Spark pour le traitement à grande échelle

— Déploiement dans le cloud (AWS, GCP, Azure)

— Ajout de modèles prédictifs pour l'estimation des prix

— Mise en place d'alertes en temps réel sur les annonces intéressantes

— Exposition des données via une API REST avec FastAPI

Références

1. Documentation Apache Airflow : https://airflow.apache.org/
2. Documentation MinIO : https://min.io/docs/
3. Documentation Metabase : https://www.metabase.com/docs/
4. Documentation Docker : https://docs.docker.com/
5. Documentation PostgreSQL : https://www.postgresql.org/docs/
6. "Data Engineering with Python" – Paul Crickard
7. "Big Data Architecture" – Nathan Marz